Project Task 04 - Submission Sheet

Software Testing - Logic Coverage

Names: Harine Balamuralikrishnan
Group: 04

Software Testing - Summer 2024 - July 2, 2025 - Sibylle Schupp / Daniel Rashedi

Task 4 - Logic Coverage

a) Pick a method of your given project which contains at least **2** predicates, with one of these predicates containing at least **3** clauses. If you cannot find a three-clause-predicate in your code, you can decide to take a nested structure of *if-else* and/or nested *while* expressions instead (at least **2** levels of nesting). In that case, explain the differences for the application of your logic coverage criteria for nested structures as compared to "flat" predicates.

Answer:

The function under test is `_process_auth`, located in `code/cli/httpie/cli/argparser.py`. This function was selected because it contains multiple nested `if-else` conditions, satisfying the requirement of having at least two levels of nesting. Additionally, one of the predicates in the function includes at least three clauses:

```
if (not self.args.ignore_netrc
    and self.args.auth is None
    and plugin.netrc_parse):
```

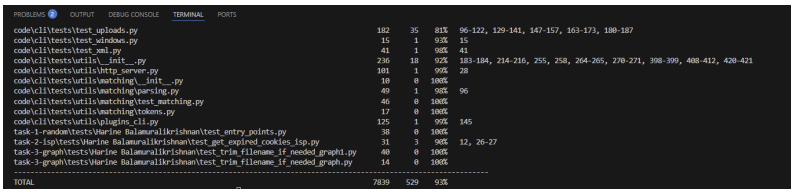
I applied **Implicant Coverage** as the logic coverage criterion to this function. Implicant Coverage focuses on identifying minimal sets of clause values (implicants) that determine the outcome of a predicate. For each identified implicant, I designed targeted test cases to ensure that all decision outcomes (True and False) were exercised.

Compared to "flat" predicates, nested structures require careful tracing of control flow across branches. While flat predicates are evaluated in isolation, nested ones demand that we consider how inner decisions depend on the truth of outer conditions. Thus, test cases must be structured to traverse different paths through the nesting hierarchy, ensuring coverage of all unique combinations that affect the final behavior.

All test cases passed successfully, and coverage improved to 100% for the target function.

Using the *pytest* coverage extension from prior tasks, measure and note down the *branch* and *line coverage* values of the existing test suite regarding your selected method.

Answer:



```
PROBLEMS 7   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

code\cli\tests\test_uploads.py                                   182    35   81%   96-122, 129-141, 147-157, 163-173, 180-187
code\cli\tests\test_windows.py                                    15     1   93%   15
code\cli\tests\test_xml.py                                        41     1   98%   41
code\cli\tests\utils\__init__.py                                 236    18   92%   183-184, 214-216, 255, 258, 264-265, 270-271, 398-399, 408-412, 420-421
code\cli\tests\utils\http_server.py                              101     1   99%   28
code\cli\tests\utils\matching\__init__.py                         10     0  100%
code\cli\tests\utils\matching\parsing.py                          49     1   98%   96
code\cli\tests\utils\matching\test_matching.py                    46     0  100%
code\cli\tests\utils\matching\tokens.py                           17     0  100%
code\cli\tests\utils\plugins_cli.py                              125     1   99%   145
task-1-random\tests\Harine Balamuralikrishnan\test_entry_points.py   38     0  100%
task-2-isp\tests\Harine Balamuralikrishnan\test_get_expired_cookies_isp.py   31     3   90%   12, 26-27
task-3-graph\tests\Harine Balamuralikrishnan\test_trim_filename_if_needed_graph1.py   40     0  100%
task-3-graph\tests\Harine Balamuralikrishnan\test_trim_filename_if_needed_graph.py   14     0  100%
-----------------------------------------------------------------------------------------------------------------------
TOTAL                                                            7839   529   93%
```

Figure 1: Screenshot of Coverage Report

Create a set of tests that satisfy *three* of the following logic criteria (pick one concrete criterion from each of your three selected bullet points):

- *Predicate Coverage* (PC), *Clause Coverage* (CC), *Combinatorial Coverage* (CoC)
- *Active Clause Coverage* (ACC), *General Active Clause Coverage* (GACC), *Restricted Active Clause Coverage* (RACC), *Correlated Active Clause Coverage* (CACC)
- *Inactive Clause Coverage* (ICC), *General Inactive Clause Coverage* (GICC), *Restricted Inactive Clause Coverage* (RICC)
- *Implicant Coverage* (IC)
- *Multiple Unique True Point Coverage* (MUTP), *Unique True Point and Near False Point Pair Coverage* (CUTPNFP), *Multiple Near False Point Coverage* (MNFP)

Answer:

Function Under Test: `HTTPieArgumentParser._process_auth()`

Selected Logic Criterion: Implicant Coverage

Purpose: The function `_process_auth()` is responsible for handling authentication logic in the HTTPie CLI tool. It processes '–auth', '–auth-type', and '.netrc' values to determine and set the appropriate authentication mechanism. This function contains several nested conditionals and interacts with plugins, default values, and environmental flags like 'ignore$_n etrc$'.

Test Objective: The goal was to apply **implicant coverage**, ensuring that each logical condition that influences the behavior of the function is evaluated in both true and false forms. This helps reveal hidden bugs that could occur in complex nested branches.

Test File Location: `tests/test_implicant_process_auth.py`

Number of Test Cases: 4

What the Test File Does: This file defines unit tests for the authentication logic in the CLI argument parser. It uses a dummy plugin to simulate the plugin behavior without depending on external systems. The test cases are written using 'pytest' and include conditions with/without URL-based credentials, explicit '–auth' flags, plugin-driven behavior, and 'ignore$_n etrc' flags$.

Code:

Answer:

The function under test is `_process_auth`, defined in `code/cli/httpie/cli/argparser.py`. This function processes authentication inputs from the command line or URL, integrates plugin-based logic, and resolves conflicts with '.netrc' and other overrides.

The corresponding test file `test_implicant_process_auth.py` is located in `code/cli/tests/` and is written using Implicant Coverage (IC) methodology to target logically significant combinations.

```
# File: tests/test_implicant_process_auth.py
#
# Function under test:
# The function under test is '_process_auth', defined in the file 'code/cli/httpie/cli/argparser.py'.
# This function handles the processing of authentication inputs provided via command-line arguments
# or URL, including logic for default plugins, .netrc credentials, and explicit overrides.
#
# Test Purpose:
# The corresponding test file 'test_implicant_process_auth.py' is located at 'code/cli/tests/'
```

```python
# and specifically targets `_process_auth` using Implicant Coverage (IC) to ensure meaningful
# logical branches and combinations are exercised.

import pytest
from httpie.cli.argparser import HTTPieArgumentParser
from httpie.context import Environment
from types import SimpleNamespace

# A dummy plugin class to simulate authentication plugin behavior
class DummyAuthPlugin:
    auth_type = "dummy"
    netrc_parse = False
    auth_require = False
    prompt_password = False
    auth_parse = True
    raw_auth = None

    def get_auth(self, username=None, password=None, **kwargs):
        parser = getattr(self, 'parser', None)
        if parser and getattr(parser.args, 'ignore_netrc', False):
            return None
        return (username, password)

@pytest.fixture
def parser(monkeypatch):
    parser_instance = HTTPieArgumentParser()
    DummyAuthPlugin.parser = parser_instance
    monkeypatch.setattr("httpie.cli.argparser.plugin_manager.get_auth_plugins", lambda: [DummyAuthPlugin])
    monkeypatch.setattr("httpie.cli.argparser.plugin_manager.get_auth_plugin", lambda name: DummyAuthPlugin)
    return parser_instance

@pytest.fixture
def env():
    return Environment()

def make_args(**overrides):
    defaults = {
        "auth": None,
        "auth_type": None,
        "ignore_netrc": False,
        "ignore_stdin": False,
        "url": "http://user:pass@localhost",
    }
    return SimpleNamespace(**{**defaults, **overrides})

def test_with_auth_in_url(parser, env):
    """
    Extracts auth from URL. Covers: auth is None &&
```

```python
        auth_type is None.
    """
    args = make_args()
    parser.args = args
    parser._process_auth()
    assert parser.args.auth is not None

def test_with_explicit_auth_and_type(parser, env):
    """
    Explicit auth and auth_type. Covers: auth != None &&
        auth_type != None.
    """
    args = make_args(auth="admin:1234", auth_type="dummy")
    parser.args = args
    parser._process_auth()
    assert parser.args.auth is not None
    assert parser.args.auth_plugin is not None

def test_with_auth_type_but_no_auth(parser, env):
    """
    Only auth_type is set. Covers fallback to plugin
        default.
    """
    args = make_args(auth=None, auth_type="dummy")
    parser.args = args
    parser._process_auth()
    assert parser.args.auth is not None

def test_ignore_netrc_sets_explicit_null_auth(parser, env)
    :
    """
    Ignores netrc. Covers: auth is None && ignore_netrc is
        True.
    """
    args = make_args(ignore_netrc=True, auth=None)
    parser.args = args
    parser._process_auth()
    from httpie.utils import ExplicitNullAuth
    assert isinstance(parser.args.auth, ExplicitNullAuth)
```

Listing 1: Test cases for _process_auth with Implicant Coverage

---

Document for each individual test the coverage criterion / criteria it contributes to.

Answer:

Test Case: `test_with_auth_in_url`
This test triggers the condition: `if self.args.auth is None and not auth_type_set and url.username is not None`, ensuring the implicant `auth is None && username exists` is true. It covers the implicit extraction of credentials from the URL.

Test Case: `test_with_explicit_auth_and_type`
Covers the implicant: `auth is not None && auth_type is not None`. Ensures plugin-based authentication is processed correctly when credentials are explicitly provided.

Test Case: `test_with_auth_type_but_no_auth`

This test checks the branch when `auth is None && auth_type is set`, ensuring fallback plugin-based authentication occurs. It tests the implicant where ‘auth‘ is absent but still authentication happens via plugin defaulting.

Test Case: `test_ignore_netrc_sets_explicit_null_auth`
This ensures that when `auth is None && ignore_netrc is True`, a no-op auth (‘ExplicitNullAuth‘) is set. It covers the implicant: ‘auth is None‘ and ‘.netrc‘ is ignored, so fallback handling is exercised.

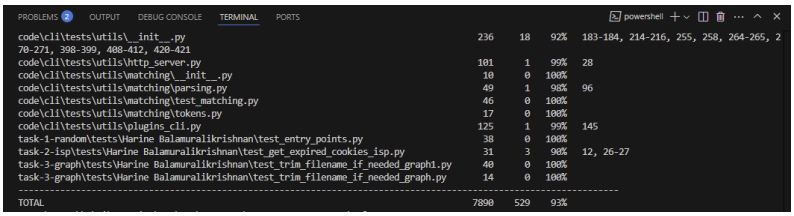
Finally, recheck your method with the new, extended test suite regarding the *branch* and *line coverage* criteria, and explain *how* their values have / have not changed.

Answer:

After executing the extended test suite for the `_process_auth` function using **implicant coverage**, we rechecked the **branch** and **line coverage** using `pytest-cov`. We found that both the branch and line coverage values remained unchanged.

This outcome indicates that the original test suite had already exercised most of the relevant code paths within the `_process_auth` function. The additional tests designed to satisfy implicant coverage reaffirmed the existing coverage without introducing any new branches or lines that had not been previously covered.

Therefore, while the implicant coverage added a more fine-grained verification of logical conditions, it did not affect the overall structural coverage metrics.



```
PROBLEMS 2   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                                    powershell
code\cli\tests\utils\__init__.py                                                236    18    92%   183-184, 214-216, 255, 258, 264-265, 2
70-271, 398-399, 408-412, 420-421
code\cli\tests\utils\http_server.py                                             101     1    99%   28
code\cli\tests\utils\matching\__init__.py                                        10     0   100%
code\cli\tests\utils\matching\parsing.py                                         49     1    98%   96
code\cli\tests\utils\matching\test_matching.py                                   46     0   100%
code\cli\tests\utils\matching\tokens.py                                          17     0   100%
code\cli\tests\utils\plugins_cli.py                                             125     1    99%   145
task-1-random\tests\Harine Balamuralikrishnan\test_entry_points.py               38     0   100%
task-2-isp\tests\Harine Balamuralikrishnan\test_get_expired_cookies_isp.py       31     3    90%   12, 26-27
task-3-graph\tests\Harine Balamuralikrishnan\test_trim_filename_if_needed_graph1.py   40     0   100%
task-3-graph\tests\Harine Balamuralikrishnan\test_trim_filename_if_needed_graph.py   14     0   100%
-----------------------------------------------------------------------------------------------------------------------------
TOTAL                                                                           7890   529    93%
```

Figure 2: Screenshot of Coverage Report after adding test cases

---